Logos2 系列 FPGA 算术处理模块（APM）用户指南

(Version0.2)

（2020.9.27）

深圳市紫光同创电子有限公司

版权所有侵权必究

文档版本修订记录

版本号	发布日期	修订记录
V0.1	2019/12/3	初始版本
V0.2	2020/9/27	修订了表 2、表 5、表 8 的部分配置位设置，对附录部分示例增加说明

名词术语解释

名词术语	英文全称	名词术语中文意义
APM	Arithmetic Process Module	算术处理单元
SRB	Signal Relay Block	信号转接单元
PDS	Pango Design Suite	紫光同创电子设计工具

目录

1. 总体介绍.....1
2. 功能描述.....2
2.1. APM 结构说明.....2
2.2. GTP 说明.....3
2.3. APM 工作模式.....10
2.4. 具体功能描述.....12
3. 附录.....15

图目录

图 1 APM 整体功能框图 2
图 2 结构框图 4
图 3 乘法模式应用示意图 10
图 4 带预加的乘法模式应用示意图 11
图 5 乘法模式的典型时序图 11
图 6 通用乘加模式应用示意图 9
图 7 通用乘加模式的典型时序 9
图 8 乘累加模式应用示意图 12
图 9 带预加的乘累加应用示意图 13
图 10 乘累加模式的典型时序 13
图 11 Systolic FIR 功能示意图 14

表目录

表 1 GTP_APM_E2 乘法模式 I/O 列表说明 4
表 2 MODE 端口功能说明 6
表 3 MODEIN 功能列表 7
表 4 GTP_APM_E2 参数列表说明 7
表 5 典型 FIR 滤波器实现 11
表 6 Rounding 详细功能列表 12
表 7 Rounding 各功能配置列表 13
表 8 时分复用详细表格 13
表 9 APM 级联可实现功能描述 15

1. 总体介绍

Logos2 系列产品支持 APM（Arithmetic Process Module，算术处理单元），为 Logos2 系列产品提供了高效的数字信号处理能力。

APM 在 Logos2 系列产品中按列分布，其主要特性有：

- 有符号数乘法器 25*18;无符号乘通过高位赋 0 实现
- 所有的计算及输出结果均为有符号数，已包含符号位
- 支持 1 个 48bits 加/减/累加运算或 2 个 24bits 运算
- Pre-add 为 25bits
- 独立可选的 CE 和 RST
- 支持输入级联
- 支持输出级联
- 控制/数据信号流水线
- 支持动态模式切换
- 支持 Rounding 功能

APM的实现可通过深圳市紫光同创电子有限公司的软件Pango Design Suite（后文简称为PDS）来完成。为方便用户使用，Pango Design Suite内嵌的IP Compiler工具提供IP的生成。PDS软件中集成IP用户指南，具体IP的使用可在选择APM的使用模式后查阅相应的IP用户指南。

2. 功能描述

2.1. APM 结构说明

APM 主要由 I/O Unit（I/O 单元）、Preadd Unit（预加单元）、Mult Unit（乘法单元）和 Postadd Unit（累加单元）四个功能单元组成。APM 的整体功能框图如下所示。

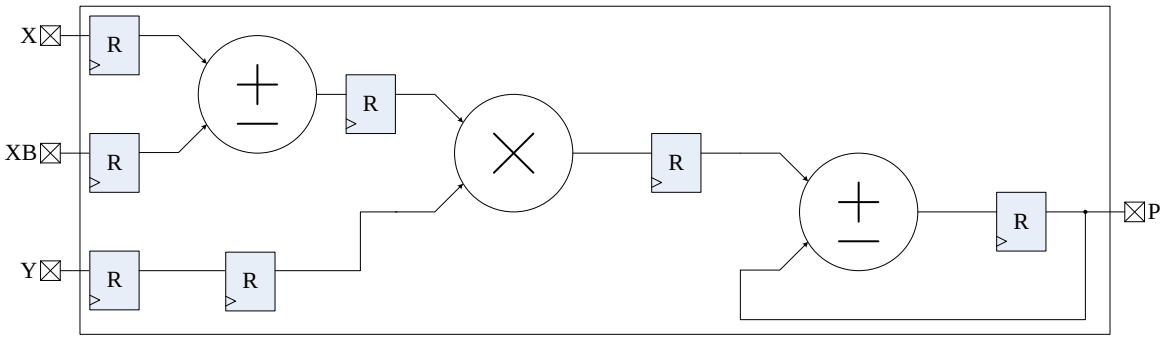


图 1 APM 整体功能框图

1. I/O 单元

- 实现数据输入/输出寄存。
- 各个寄存器的 CE 信号（高有效）和 RST 信号（高有效）都有其各自独立的 CE 和 RST 输入来控制。RST 异步/同步性质由共享参数 ASYNC 决定。
- 模式端口按控制功能归属分成 MODEIN[4:0], MODEY[2:0], MODEZ[3:0]。每组模式端口控制由各自的独立的 REG，RST，CE 参数控制。
- APM 级联用于实现 FIR 以及高位宽乘法器如 49*35。
- 实现对常数输入和重复的符号位简化接线。
- X 端口支持 30 位二则运算，端口 XB[24:0]可支持 PREADD_MULTADD 运算，MODEIN 控制端用于支持时分复用的功能。

2. 预加单元

- 硬件实现为 25+/-25，输出 25 位结果；或两个 12+/-12 ，各输出 12 位结果(当 SIMD=1)。
- 预加功能可被旁路（PREADD=0）。

3. 乘法单元

- 实现一个 25*18，结果带符号扩展为 48 位；或两个 12*9（当 SIMD=1），12*9 结果带符

号扩展为 24 位。

- 操作数均为有符号数。

4. 累加单元

- 累加单元支持实现一个 48 位加法；或两个 24 位加法，减法运算 P=X-Y 转换成加法 P=X+~Y+1 来运算。
- POSTADD 可被旁路（当 POSTADD=0），此时 APM 输出乘法器结果。

2.2. GTP 说明

GTP_APM_E2 单元集成了 APM 的各种工作模式，用户可通过对接口和参数的配置，实现各种模式的选择和 APM 功能实现。

1. GTP 结构框图

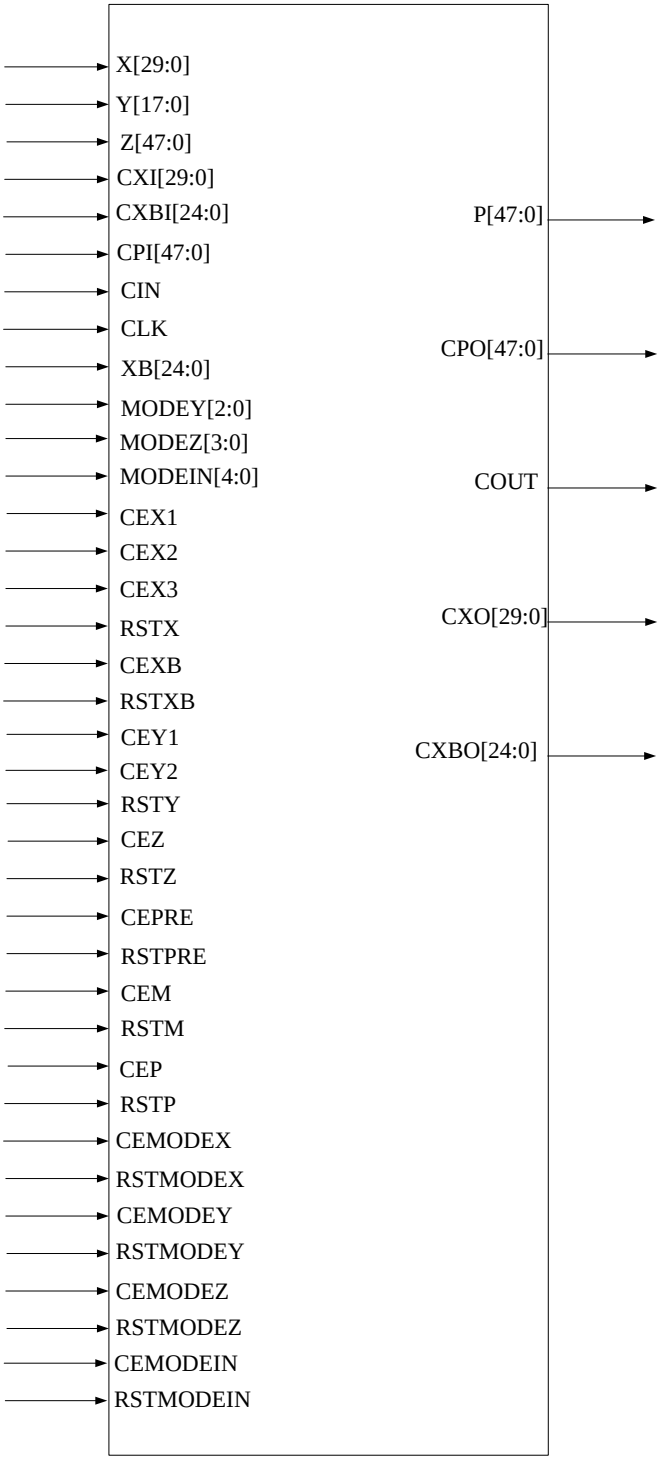


图 2 结构框图

2. 端口说明

表 1 GTP_APM_E2 乘法模式 I/O 列表说明

端口名	输入/输出	描述
X[29:0]	输入	并行数据输入 X

端口名	输入/输出	描述
CXI[29:0]	输入	级联 X 输入
CXBI[24:0]	输入	级联 H 输入
XB[24:0]	输入	并行数据输入 XB
Y[17:0]	输入	并行数据输入 Y
Z[47:0]	输入	并行数据输入 Z
CPI[47:0]	输入	级联 P 输入
CIN	输入	级联 CIN 输入
MODEY[2:0]	输入	APM 动态 Y 端控制操作符
MODEZ[3:0]	输入	APM 动态 Z 端控制操作符
MODEIN[4:0]	输入	APM 动态输入控制操作符
CLK	输入	时钟
CEX1	输入	时钟 X REG1 使能信号
CEX2	输入	时钟 X REG2 使能信号
CEX3	输入	时钟 X REG3 使能信号
RSTX	输入	高有效的 X 复位信号
CEXB	输入	时钟 X B 使能信号
RSTXB	输入	高有效的 XB 复位信号
CEY1	输入	时钟 YREG1 使能信号
CEY2	输入	时钟 Y REG2 使能信号

端口名	输入/输出	描述
RSTY	输入	高有效的 Y 复位信号
CEZ	输入	时钟 Z 使能信号
RSTZ	输入	高有效的 Z 复位信号
CEPRE	输入	时钟 PRE 使能信号
RSTPRE	输入	高有效的 PRE 复位信号
CEM	输入	时钟 M 使能信号
RSTM	输入	高有效的 M 复位信号
CEP	输入	时钟 P 使能信号
RSTP	输入	高有效的 P 复位信号
CEMODEIN	输入	时钟 MODEIN 使能信号
RSTMODEIN	输入	高有效的 MODEIN 复位信号
CEMODEY	输入	时钟 MODEY 使能信号
RSTMODEY	输入	高有效的 MODEY 复位信号
CEMODEZ	输入	时钟 MODEZ 使能信号
RSTMODEZ	输入	高有效的 MODEZ 复位信号
P[47:0]	输出	并行数据输出
CPO[47:0]	输出	级联 P 输出
COUT	输出	级联 CIN 输出
CXO[29:0]	输出	级联 X 输出

端口名	输入/输出	描述
CXBO[24:0]	输出	级联 XB 输出

3. 模式说明

控制 APM 工作模式的为 MODEIN[4:0]，MODEY[2:0]，MODEZ[3:0]三组信号，它们的端口说明如表 2 所示，其中 MODEIN 详细的列表请见表 3(考虑预加器一路选择 XB 端数据)。两级的 X1,X2 和 Y1，Y2 两级寄存器主要考虑时分复用的功能下各个功能数据延迟的一致性,多出的一级寄存器可以补偿 PIPELINE 寄存器的延迟。

表 2 MODE 端口功能说明

信号	说明
MODEIN[4]	0：Y2 1：Y1
MODEIN[3]	Preadder 加减选择 0：加 1：减
MODEZ[3]	对 Zmux 输出取负值, 1 有效
MODEZ[2:0]	0：零 1：累加器反馈 2：Z 输入端 3：上一级 APM 结果级联输出 4：上一级 APM 结果级联输出右移 17 位 5：上一级 APM 结果级联输出右移 24 位 6：上一级 APM 结果级联输出右移 16 位 7：上一级 APM 结果级联输出右移 8 位
MODEY[2]	对 Ymux 输出取负值，1 有效
MODEY[1：0]	0：零 1：乘法器输出 2：累加器反馈 3：宽位宽数据{X[29：0],Y[17：0]}

表 3 MODEIN 功能列表

MODEIN [3]	MODEIN [2]	MODEIN [1]	MODEIN [0]	"PREADD"	X_MULT

MODEIN [3]	MODEIN [2]	MODEIN [1]	MODEIN [0]	"PREADD"	X_MULT
x	x	0	0	0	zero
x	x	0	1	0	zero
x	x	1	0	0	X2
x	x	1	1	0	X1
0	0	0	0	1	zero
0	0	0	1	1	zero
0	0	1	0	1	X2
0	0	1	1	1	X1
0	1	0	0	1	XB
0	1	0	1	1	XB
0	1	1	0	1	X2+XB
0	1	1	1	1	X1+XB
1	0	0	0	1	zero
1	0	0	1	1	zero
1	0	1	0	1	X2
1	0	1	1	1	X1
1	1	0	0	1	-XB
1	1	0	1	1	-XB
1	1	1	0	1	X2-XB

MODEIN [3]	MODEIN [2]	MODEIN [1]	MODEIN [0]	"PREADD"	X_MULT
1	1	1	1	1	X1-XB

4. 参数说明

表 4 GTP_APM_E2 参数列表说明

参数名	有效值	默认值	功能描述
USE_POSTADD	0,1	0	Post add 功能使能
USE_PREADD	0,1	0	Pre add 功能使能
CXO_REG	0,1,2,3	0	X 级联输出寄存器延迟
X_REG	0,1,2,3	0	X 端口寄存器使能
XB_REG	0,1	0	XB 端口寄存器使能
Y_REG	0,1,2,3	0	Y 端口寄存器使能
Z_REG	0,1	0	Z 端口寄存器使能
PREADD_REG	0,1	0	预加寄存器使能
MULT_REG	0,1	0	Mult 寄存器使能
P_REG	0,1	0	Post add 寄存器使能
MODEIN_REG	0,1	0	MODEIN 寄存器使能
MODEY_REG	0,1	0	MODEY 寄存器使能
MODEZ_REG	0,1	0	MODEZ 寄存器使能
X_SEL	0,1	0	X 输入选择
XB_SEL	0,1,2,3	0	X 向后传递 mux 选择

ASYNC_RST	0,1	0	异步复位使能
USE_SIMD	0,1	0	模式选择
P_INIT0	48'h0-48'hfff	48'h0	P0 输入
P_INIT1	48'h0-48'hfff	48'h0	P1 输入
ROUNDMODE_SEL	0,1	0	Round 模式选择
CPO_REG	0,1	0	P0，PCO 输出寄存器使能
USE_ACCLOW	0,1	0	ACC 仅使用低 17 位反馈使能
CIN_SEL	0,1	0	Postadder 选择 PCI 作为输入
GRS_EN	"TRUE","FALSE"	"TRUE"	全局复位使能

2.3. APM 工作模式

2.3.1. 乘法模式

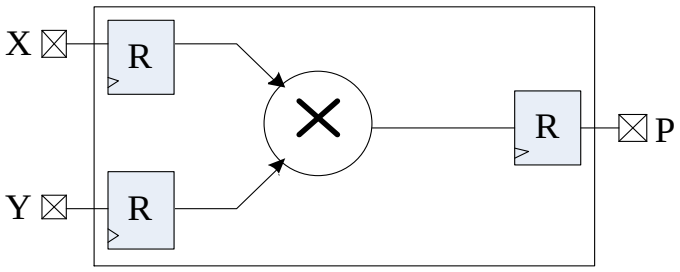


图 3 乘法模式应用示意图

如图3所示，APM配置成乘法模式时，其等效算术表达式为：

$$P=X*Y$$

APM配置成乘法模式时主要特性有：

- 每个APM可支持一个25*18或两个12*9运算
- 输入/输出寄存器可选

使能APM中的Preadd Unit后，APM可通过XB端口配置成带预加的乘法模式，其算术表达式为：

$$P=Y*(X+/-XB)$$

该模式下，每个APM可实现两个(12+/-12)*9运算，或1个(25+/-25)*18运算，支持动/静态预加/减控制。除了可选的输入/输出寄存器，带预加的乘法模式还可选择使能内部流水寄存器，其应用示意图如图4所示：

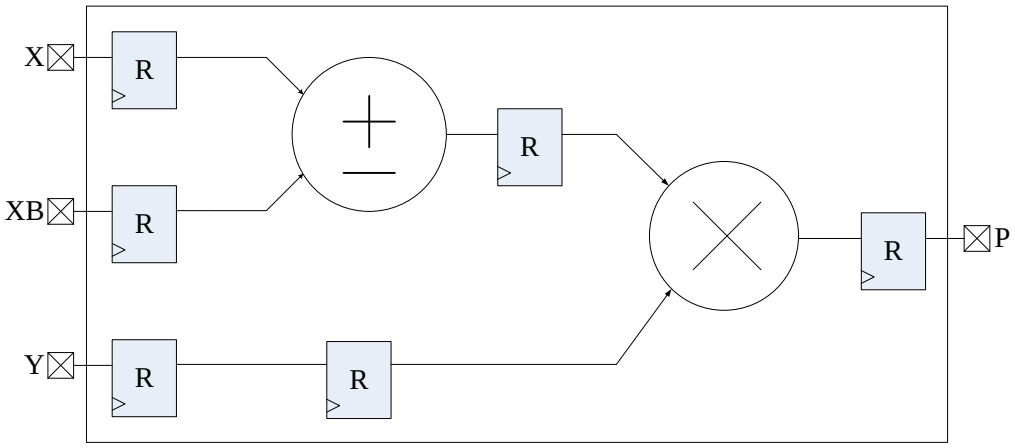


图 4 带预加的乘法模式应用示意图

乘法模式的典型时序如下，其中P=Y(X+XB)，P在CEM有效时才能输出。P端对应X,Y端输入数据的计算结果在下一个时钟上升沿输出。如图5所示。

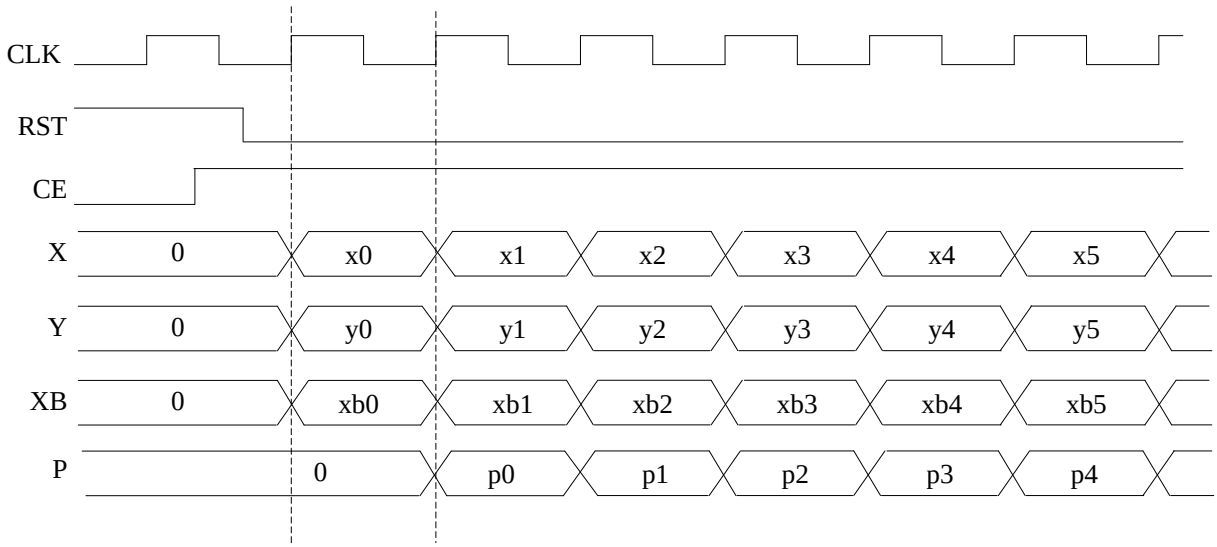


图 5 乘法模式的典型时序图

2.3.2. 通用乘加模式

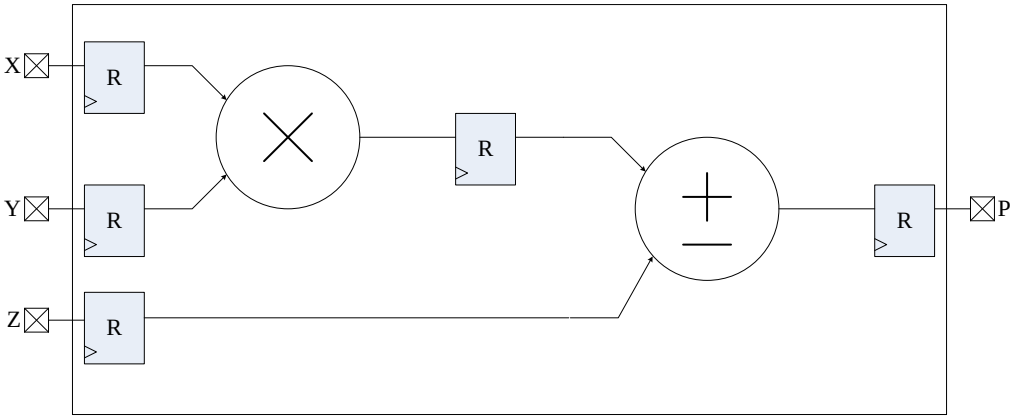


图 6 通用乘加模式应用示意图

如图6所示，APM配置成通用乘加模式时，其算术表达式为：

$$P=X*Y +/- Z$$

通用乘加模式的主要特性有：

- 每个APM可实现一个25*18+/-48或两个12*9+/-24运算
- 支持有符号数，加减都可动/静态控制
- 可选的输入/输出寄存器

使用GTP_APM_E2单元进行例化。通用乘加模式的典型时序如下，其中P=X*Y+Z，P在CEP有效时才能输出。

当MODEY[2]==1，乘法器输出取反，等价于乘法器取负，当MODEZ[3] == 1，Zmux输出取反，等价于Zmux取负。

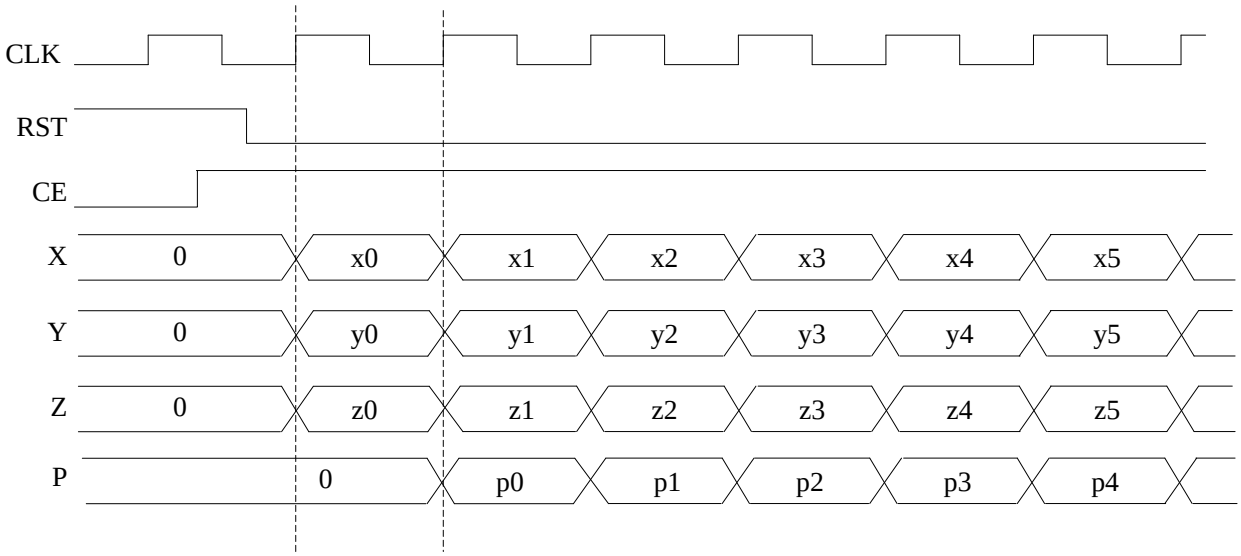


图 7 通用乘加模式的典型时序

2.3.3. 乘累加模式

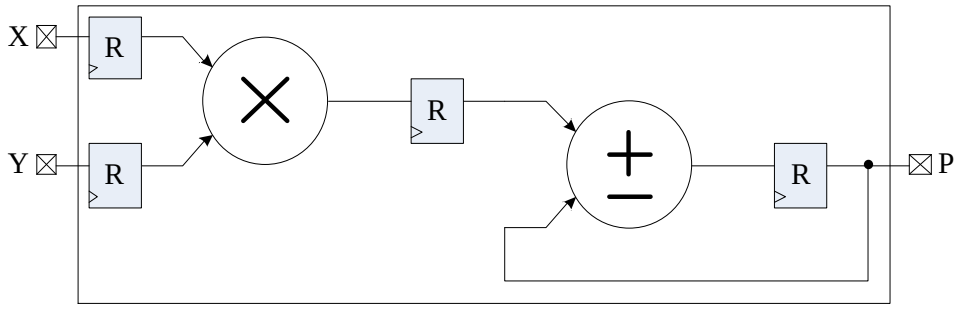


图 8 乘累加模式应用示意图

如图8所示，APM配置成乘累加模式时，其等效算术表达式为：

$$P= P +/- X*Y$$

乘累加模式的主要特性有：

- 一个APM可实现一个25*18的乘累加运算(P为48bit)或两个12*9的乘累加运算(P为24bit)

- 支持有符号数，支持动/静态累加/减控制
- 可选的输入/输出寄存器
- P值可预置
- 支持RELOAD功能

使能 APM 中的 Preadd Unit 后，APM 可配置成带预加的乘累加模式，其算术表达式为：

P=P+/- Y*(X+/-XB)

带预加的乘累加模式有两级内部流水寄存器可供选择，其应用示意图如图 9 所示：

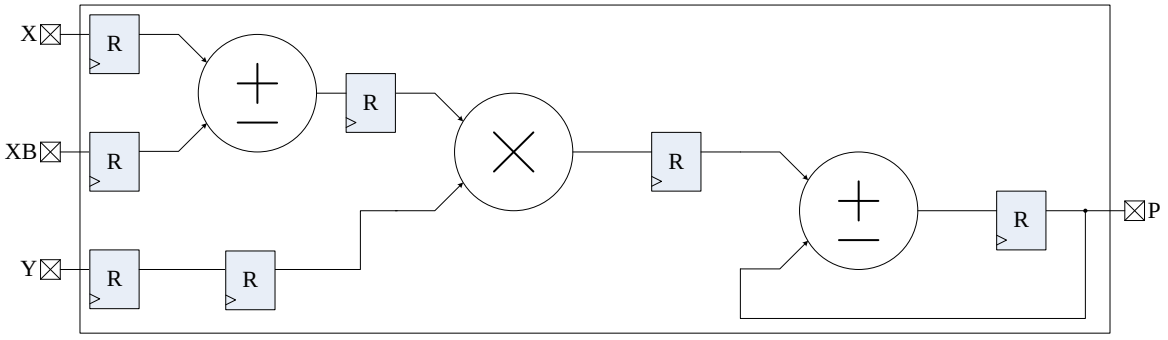


图 9 带预加的乘累加应用示意图

使用GTP_APM_E2单元进行例化。乘累加模式的典型时序如下，其中X=1，Y=1，P=P+X*Y，P在CEP有效时才能输出。

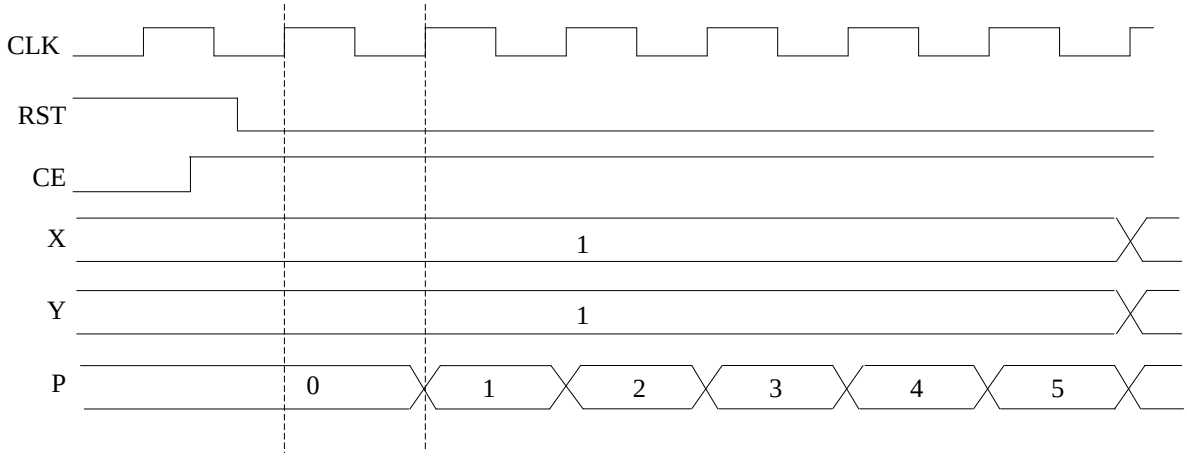


图 10 乘累加模式的典型时序

2.3.4. FIR 模式

通过APM级联也可实现FIR运算，典型的FIR filter可以描述为以下表达式：

$$y_n = \sum_{i=0}^{N-1} x_{n-i} h_i = x_n h_0 + x_{n-1} h_1 + \ldots + x_{n-N+1} h_{N-1}$$

x为输入的数据流，y为输出的数据流，h为系数。图10为采用24个APM级联实现的24阶Systolic FIR运算的功能实现图。

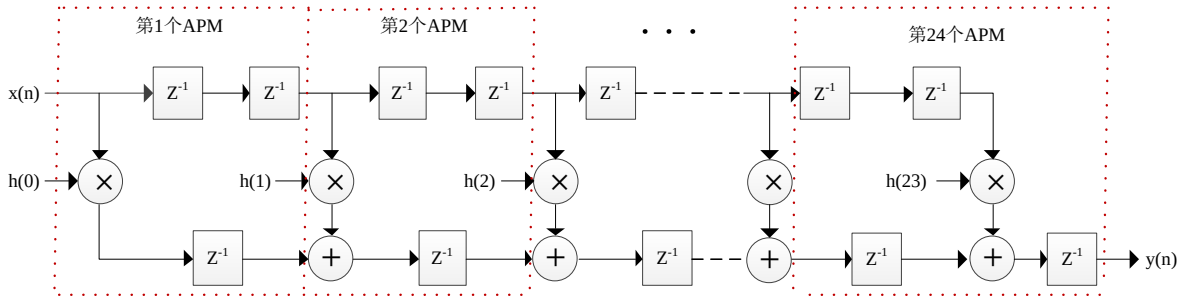


图 11 Systolic FIR 功能示意图

通过对GTP_APM_E2单元的配置，可以实现多种形式的FIR运算，具体的实现类型及参数配置建议如表5所示（所有FIR配置里都有POREG = 1）。

表 5 典型 FIR 滤波器实现

滤波器类型	第一级 APM	中间级 APM	最后一级 APM
Common setting:	POREG=1		
Systolic FIR	XOREG=2 XREG=1 XSEL=0	XOREG=2 XREG=1 XSEL=1	XREG=1 XSEL=1
2-Channel Systolic FIR	XOREG=3 XREG=1 XSEL=0	XOREG=3 XREG=1 XSEL=1	XREG=1 XSEL=1
Transposed FIR	XOREG=1 XREG=1 XSEL=0	XOREG=0 XREG=0 XSEL=1	XREG=0 XSEL=1
Symmetrical FIR (偶数 tap)	PREADD=1 XOREG=2 XREG=1 XSEL=0 XBSEL=2’b11 HREG=0	PREADD=1 XOREG=2 XREG=1 XSEL=1 XBSEL=2’b11 HREG=0	PREADD=1 XOREG=2 XREG=1 XSEL=1 XBSEL[1]=1’b1
Symmetrical FIR (奇数 tap)	PREADD=1 XOREG=2 XREG=1 XSEL=0 XBSEL=2’b11 HREG=0	PREADD=1 XOREG=2 XREG=1 XSEL=1 XBSEL=2’b11 HREG=0	PREADD=0 XOREG=1 XREG=1 XSEL=1 XBSEL[1]=1’b1
2-Channel Symmetrical FIR （偶数 tap）	PREADD=1	PREADD=1	PREADD=1

滤波器类型	第一级 APM	中间级 APM	最后一级 APM
	XOREG=3 XREG=1 XSEL=0 XBSEL=2'b11 HREG=1	XOREG=3 XREG=1 XSEL=1 XBSEL=2'b11 HREG=1	XOREG=3 XREG=1 XSEL=1 XBSEL[1]=1'b1
2-Channel Symmetrical FIR (奇数 tap)	PREADD=1 XOREG=3 XREG=1 XSEL=0 XBSEL=2'b11 HREG=1	PREADD=1 XOREG=3 XREG=1 XSEL=1 XBSEL=2'b11 HREG=1	PREADD=0 XOREG=1 XREG=1 XSEL=1 XBSEL[1]=1'b1

2.4. 具体功能描述

2.4.1. Rounding 功能

Logos2 APM支持乘法运算和乘加运算的Direct rounding和Round-toward-nearest，其详细功能如表6所示。

表 6 Rounding 详细功能列表

Rounding 功能		功能举例说明	乘法功能	乘加减功能	实现方式（N 为浮点位置）
Direct rounding	Round-ceiling	3.1,3.5,3.9>4 -3.1,-3.5,-3.9>-3	支持	支持	+2^N-1,然后取整
	Round-floor	3.1,3.5,3.9>3 -3.1,-3.5,-3.9>-4	支持	支持	直接取整
	Round-toward zero	3.1,3.5,3.9>3 -3.1,-3.5,-3.9>-3	支持	支持	对负数+2^N-1,正数直接取整
	Round-away from zero	3.1,3.5,3.9>4 -3.1,-3.5,-3.9>-4	支持	支持	对正数+2^N-1,负数直接取整
Round-toward-nearest	Symmetric Round-half-up	3.1>3; 3.5,3.9>4 -3.1>-3; -3.5,-3.9>-4	支持	支持	正数+2^（N-1）,负数+2^（N-1）-1，然后取整

Rounding 功能		功能举例说明	乘法功能	乘加减功能	实现方式（N 为浮点位置）
	Asymmetric Round-half-up	3.1>3; 3.5,3.9>4 -3.1,-3.5>-3; -3.9>-4	支持	支持	+2^（N-1）,然后取整
	Symmetric Round-half-down	3.1,3.5>3; 3.9>4 -3.1,-3.5>-3; -3.9>-4	支持	支持	正数+2^（N-1）-1,负数 +2^（N-1），然后取整
	Asymmetric Round-half-down	3.1,3.5>3; 3.9>4 -3.1>-3; -3.5,-3.9>-4	支持	支持	+2^（N-1）-1,然后取整

Logos2 APM实现不同Rounding功能的参数配置如下：

表 7 Rounding 各功能配置列表

Rounding 功能		ROUNDMODE	PINIT0	PINIT1
Direct rounding	Round-ceiling	0	2^N-1	0
	Round-floor	0	0	0
	Round-toward zero	1	0	2^N-1
	Round-away from zero	1	2^N-1	0
Round-toward-nearest	Symmetric Round-half-up	1	2^(N-1)	2^(N-1)-1
	Asymmetric Round-half-up	0	2^(N-1)	0
	Symmetric Round-half-down	1	2^(N-1)-1	2^(N-1)
	Asymmetric Round-half-down	0	2^(N-1)-1	0

为支持所有的rounding功能，累加模块需要两个48位的加法器，后一个加法器的一个输入需要由

前一个加法器输出的正负性来决定。

2.4.2. 时分复用

Logos2的APM支持时分复用功能，表8给出了时分复用功能支持的运算功能及其参数配置。

表 8 时分复用详细表格

	寄存器情况			动态控制配置			
	input reg	pipe reg	output reg	MODEIN [1]	MODEIN [2]	MODEY [1：0]	MODEZ [2：0]
MULT25	1	1	1	1	0	2’b01	3’b000
PREADD_MULT25	1	1	1	1	1	2’b01	3’b000
PRE_TADD25	1	1	1	1	0	2’b01	3’b010
PRE_MULTADD25	1	1	1	1	1	2’b01	3’b010
宽位ADD/SUB	2	0	1	0	0	2’b11	3’b010

注：

1．做时分复用功能时，APM可支持功能都配置成三级寄存器。

2．input_reg包括所有数据通路和控制信号的输入寄存器，其实X和Y两路做时分复用时，需要有两个输入寄存器延迟，即X[Y]REG[1:0]要设置为2’b11。时分复用功能中pipe_reg即为MREG，output_reg即为PREG。

3．到达POSTADD的数据延迟是2个CLK延时，MODEY[1:0]在APM中只有1个输入寄存器。需错开一个clk延时控制或者MODEY[1:0]在APM外部补偿一个CLK延时。

4．Z路径和MODEZ的延迟都为1个CLK延时，与X，Y路径上的数据错开一个延时match，或者在APM外部补充一个CLK延时。

5. MODEIN [0], MODEIN [4]配置固定为1，POSTADD固定使能，时分复用中若有PREADD功能，PREADD固定使能，并且PREREG不使能，若没有PREADD可关闭。

6．PREADD_MULT25和PRE_MULTADD25没有PREREG，无法达到最高的频率。

3. 附录

级联可实现功能

通过对 APM 级联可以获得更高位宽运算，下表描述了 APM 级联可实现的功能。下表为举例，仅供参考。

表 9 APM 级联可实现功能描述

APM 级联	可实现功能
2 个 apm 级联	2 个 12*9+12*9
	2 个(12+12)*9+(12+12)*9
	输出位宽为 65 位的 MULTACC25
	输出位宽为 96 位的 MULTACC25
	输出位宽为 96 位的 ADD/ACC(加法)/
	25*18+25*18
	(25+25)*18+(25+25)*18
	输出位宽为 65 位的 PREADD_MULTACC25
	输出位宽为 96 位的 PREADD_MULTACC25
	49*18(只使用 PI 右移 24 位功能)
	42*18(只使用 PI 右移 17 位功能)
	25*35(只使用 PI 右移 17 位功能)
	25*34(只使用 PI 右移 16 位功能)
	(25+25)*35
3 个 apm	2 个输出位宽为 24 位的 MAC+(12*9+12*9)

级联	2 个输出位宽为 24 位的 MAC+((12+12)*9+(12+12)*9)
	输出为 48 位的 MAC+(25*18+25*18)
	输出为 48 位的 MAC+((25+25)*18+(25+25)*18)
	59*18(只使用 PI 右移 17 位功能)
	25*52(只使用 PI 右移 17 位功能)
	25*50(只使用 PI 右移 16 位功能)
	73*18(只使用 PI 右移 24 位功能)
	MAC+42*18
	MAC+25*35
	MAC+(25+25)*35
4 个 apm 级联	2 个(12*9+12*9)+(12*9+12*9)
	2 个((12+12)*9+(12+12)*9)+((12+12)*9+(12+12)*9)
	输出为 65 位的 MAC+(25*18+25*18)
	输出为 65 位的 MAC+((25+25)*18+(25+25)*18)
	(25*18+25*18)+(25*18+25*18)
	((25+25)*18+(25+25)*18)+((25+25)*18+(25+25)*18)
	35*25+35*25(只使用 PI 右移 17 位功能)
	42*18+42*18(只使用 PI 右移 17 位功能)
	25*34+25*34(只使用 PI 右移 16 位功能)
	49*18+49*18(只使用 PI 右移 24 位功能)

	42*35(只使用 PI 右移 17 位功能)	
	25*69(只使用 PI 右移 17 位功能)	
	76*18(只使用 PI 右移 17 位功能)	
	49*34(使用 PI 右移 16 和 8 位功能)	
	36*(25+25)+36*(25+25)	

使 用 GTP 例化为乘法模式

带预加的乘法器 GTP 例化模板如下：

- 通过USE_PREADD=1使能预加功能，不使能POSTADD
- 不使能X端口、Y端口等寄存器
- MODEIN=5'b00110，X_MULT=X2+XB，Preadder为+，选择Y2，由于未使能X/Y端口寄存器，X1/X2与X相当，Y1/Y2与Y相当
- MODEY=3'b000，MODEY控制POSTADD功能，此处没有使能POSTADD
- MODEZ=4'b0000，Zmux选择为0，不对Zmux输出取负
- ROUNDMODE_SEL=0，P_INIT0 =48'd0，P_INIT1=48'd0，rounding功能设置为Round-floor
- 输出结果P=(X+XB)*Y

```
GTP_APM_E2 #(
        .GRS_EN ("TRUE"),
        .USE_POSTADD (0), //不使能累加功能
        .USE_PREADD (1), //使能预加功能
        .PREADD_REG (0), //不使能预加寄存器

        .X_REG (0),
        .CXO_REG (0), // X级联输出寄存器延迟, 0/1/2/3
        .XB_REG (0),
        .Y_REG (0),
        .Z_REG (0),
        .MULT_REG (0),
        .P_REG (0),   //不使能累加寄存器
        .MODEY_REG (0),
        .MODEZ_REG (0),
        .MODEIN_REG (0),

        .X_SEL (0),
        .XB_SEL (0),
        .ASYNC_RST (0),
        .USE_SIMD (0),
        .P_INIT0 (48'd0),
```

```
        .P_INIT1 (48'd0),
        .ROUNDMODE_SEL (0),

        .CPO_REG (0), // PO,CPO不使能寄存器
        .USE_ACCLOW (0), //  累加仅使用低17bit作为反馈不使能
        .CIN_SEL (0) //  不选择PCI作为累加的输入
)
u0_GTP_APM_E2
(
        .P(P),    //Postadder resout
        .CPO(), //P cascade out
        .COUT(), //Postadder carry out
        .CXO(), //X cascade out
        .CXBO(), //X backward cascade out

        .X(X),
        .CXI(), //X cascade in
        .CXBI(), //X backward cascade in
        .Y(Y),
        .Z(),
        .CPI(), //P cascade in
        .CIN(), //Postadder carry in
        .XB (XB),
        .MODEIN(5'b00110),
        .MODEY(3'b000),

        .MODEZ(4'b0000),
        .CLK(clk),
        .RSTX(1'b0),
        .RSTXB(1'b0),
        .RSTY(1'b0),
        .RSTZ(1'b0),
        .RSTM(1'b0),
        .RSTP(1'b0),
        .RSTPRE(1'b0),
        .RSTMODEIN(1'b0),
        .RSTMODEY(1'b0),
        .RSTMODEZ(1'b0),

        .CEX1(1'b1),
        .CEX2(1'b1),
        .CEX3(1'b1),
        .CEXB(1'b1),
        .CEY1(1'b1),
        .CEY2(1'b1),
        .CEZ(1'b1),
        .CEM(1'b1),
        .CEP(1'b1),
```

```
        .CEPRE(1'b1),
        .CEMODEIN(1'b1),
        .CEMODEY(1'b1),
        .CEMODEZ(1'b1)
    );
```

使用 GTP 例化为乘加模式

通用乘加的GTP例化模板如下：

- 通过USE_POSTADD=1使能累加功能，不使能PREADD功能
- 不使能X端口、Y端口等寄存器
- MODEIN=5'b00010，X_MULT=X2，Preadder为+（未使能PREADD功能），选择Y2，由于未使能X/Y端口寄存器，X1/X2与X相当，Y1/Y2与Y相当
- MODEY=3'b001，MODEY控制POSTADD功能，选择为乘法器输出，不对Ymux输出取负
- MODEZ=4'b0010，Zmux选择为Z输入端，不对Zmux输出取负
- ROUNDMODE_SEL=0，P_INIT0 =48'd0，P_INIT1=48'd0，rounding功能设置为Round-floor
- 输出结果为P=X*Y+Z

```
GTP_APM_E2 #(
        .GRS_EN ("TRUE"),
        .USE_POSTADD (1), //使能累加功能
        .USE_PREADD (0),   //不使能预加功能
        .PREADD_REG (0), //不使能预加寄存器

        .X_REG (0),
        .CXO_REG (0), // X级联输出寄存器延迟, 0/1/2/3
        .XB_REG (0),
        .Y_REG (0),
        .Z_REG (0),
        .MULT_REG (0),
        .P_REG (0),   //不使能累加寄存器
        .MODEY_REG (0),
        .MODEZ_REG (0),
        .MODEIN_REG (0),

        .X_SEL (0),
        .XB_SEL (0),
        .ASYNC_RST (0),
        .USE_SIMD (0),
        .P_INIT0 (48'd0),
        .P_INIT1 (48'd0),
        .ROUNDMODE_SEL (0),

        .CPO_REG (0), // PO,CPO不使能寄存器
        .USE_ACCLOW (0), //累加仅使用低17bit作为反馈不使能
```

```
        .CIN_SEL (0) //不选择PCI作为累加的输入

)
u0_GTP_APM_E2
(
        .P(P),    //Postadder resout
        .CPO(), //P cascade out
        .COUT(), //Postadder carry out
        .CXO(), //X cascade out
        .CXBO(), //X backward cascade out

        .X(X),
        .CXI(), //X cascade in
        .CXBI(), //X backward cascade in
        .Y(Y),
        .Z(Z),
        .CPI(), //P cascade in
        .CIN(), //Postadder carry in
        .XB (XB),
        .MODEIN(5'b00010),
        .MODEY(3'b001),

        .MODEZ(4'b0010),
        .CLK(clk),
        .RSTX(1'b0),
        .RSTXB(1'b0),
        .RSTY(1'b0),
        .RSTZ(1'b0),
        .RSTM(1'b0),
        .RSTP(1'b0),
        .RSTPRE(1'b0),
        .RSTMODEIN(1'b0),
        .RSTMODEY(1'b0),
        .RSTMODEZ(1'b0),

        .CEX1(1'b1),
        .CEX2(1'b1),
        .CEX3(1'b1),
        .CEXB(1'b1),
        .CEY1(1'b1),
        .CEY2(1'b1),
        .CEZ(1'b1),
        .CEM(1'b1),
        .CEP(1'b1),
        .CEPRE(1'b1),
        .CEMODEIN(1'b1),
        .CEMODEY(1'b1),
        .CEMODEZ(1'b1)
```

);

使用 GTP 例化为乘累加模式

乘累加的GTP例化模板如下：通过USE_POSTADD=1使能累加功能，不使能PREADD功能

- 不使能X端口、Y端口等寄存器
- MODEIN=5'b00010，X_MULT=X2，Preadder为+（未使能PREADD功能），选择Y2，由于未使能X/Y端口寄存器，X1/X2与X相当，Y1/Y2与Y相当
- MODEY=3'b001，MODEY控制POSTADD功能，选择为乘法器输出，不对Ymux输出取负（取负结果变为P=P-X*Y）
- MODEZ=4'b0001，Zmux选择为累加器反馈，不对Zmux输出取负（取负结果变为P=-P+X*Y）
- ROUNDMODE_SEL=0，P_INIT0 =48'd0，P_INIT1=48'd0，rounding功能设置为Round-floor
- 在乘累加模式中，需注意先将MODEZ设为4’b0000，将APM内部的累加值初始为零，再将MODEZ值设为4’b0001进行乘累加计算
- 输出结果为P=P+X*Y

```
GTP_APM_E2 #(
        .GRS_EN ("TRUE"),
        .USE_POSTADD (1), //使能累加功能
        .USE_PREADD (0),   //不使能预加功能
        .PREADD_REG (0),   //不使能预加寄存器

        .X_REG (0),
        .CXO_REG (0), // X级联输出寄存器延迟, 0/1/2/3
        .XB_REG (0),
        .Y_REG (0),
        .Z_REG (0),
        .MULT_REG (0),
        .P_REG (0),   //不使能累加寄存器
        .MODEY_REG (0),
        .MODEZ_REG (0),
        .MODEIN_REG (0),

        .X_SEL (0),
        .XB_SEL (0),
        .ASYNC_RST (0),
        .USE_SIMD (0),
        .P_INIT0 (48'd0),
        .P_INIT1 (48'd0),
        .ROUNDMODE_SEL (0),

        .CPO_REG (1), // PO,CPO使能寄存器
```

```
        .USE_ACCLOW (0), //累加仅使用低17bit作为反馈不使能
        .CIN_SEL (0) //不选择PCI作为累加的输入
)
u0_GTP_APM_E2
(
        .P(P),    //Postadder resout
        .CPO(CPO), //P cascade out
        .COUT(), //Postadder carry out
        .CXO(), //X cascade out
        .CXBO(), //X backward cascade out

        .X(X),
        .CXI(), //X cascade in
        .CXBI(), //X backward cascade in
        .Y(Y),
        .Z(),
        .CPI(), //P cascade in
        .CIN(), //Postadder carry in
        .XB (),
        .MODEIN(5'b00010),
        .MODEY(3'b001),

        .MODEZ(4'b0001),
        .CLK(clk),
        .RSTX(1'b0),
        .RSTXB(1'b0),
        .RSTY(1'b0),
        .RSTZ(1'b0),
        .RSTM(1'b0),
        .RSTP(1'b0),
        .RSTPRE(1'b0),
        .RSTMODEIN(1'b0),
        .RSTMODEY(1'b0),
        .RSTMODEZ(1'b0),

        .CEX1(1'b1),
        .CEX2(1'b1),
        .CEX3(1'b1),
        .CEXB(1'b1),
        .CEY1(1'b1),
        .CEY2(1'b1),
        .CEZ(1'b1),
        .CEM(1'b1),
        .CEP(1'b1),
        .CEPRE(1'b1),
        .CEMODEIN(1'b1),
        .CEMODEY(1'b1),
        .CEMODEZ(1'b1)
```

```
);
```